

A Whittle Wandering – Code Audit Report

1. MapBox Token Mismanagement (High Severity)

Issue: The public web app's MapBox integration has inconsistent token handling, leading to deployment issues and quick-fix hardcoding. Initially, environment variables or meta tags were intended to supply the token, but these weren't wired correctly. This resulted in "Invalid Mapbox access token" errors, eventually mitigated by embedding the token directly in code ¹. Hardcoding the token is a security and maintainability concern.

Affected Components: Frontend React app (`public-site`). In `Map.jsx`, the Mapbox access token is hardcoded (`MAPBOX_TOKEN = 'pk....'`) instead of reliably reading from config ¹. The shared util `mapboxConfig.ts` expects `process.env.MAPBOX_PUBLIC_TOKEN` ², but the build setup provided `VITE_MAPBOX_TOKEN` (for Vite) or a different name, causing a mismatch.

Impact: A missing or invalid token prevents the interactive map from rendering. This is a user-facing failure – as noted in project docs, the map was "not rendering properly" despite a "verified working" token ³. The quick fix of inlining the token solved the symptom but introduces risk of token exposure.

Recommendation: Establish a single source of truth for the MapBox token. Use consistent env var names across build and runtime. For example, define `VITE_MAPBOX_TOKEN` in the build and reference `import { meta.env.VITE_MAPBOX_TOKEN } in the React code (or inject via an HTML meta tag, with proper secure handling) rather than using process.env on the client. Remove the hardcoded token from source; instead, pass it through a secure configuration (perhaps set as a meta tag by the server or injected at build time). This ensures the token can be rotated without code changes and isn't exposed in version control. Also add graceful error handling for token load failures (e.g., user-friendly message if token is missing) instead of silently failing.`

Evidence: The Map component explicitly hardcodes and logs a token ¹, which was a response to prior initialization errors. This indicates the environment injection was not working, likely due to inconsistent naming or missing build config. Ensuring the frontend and build pipeline use the same `VITE_MAPBOX_TOKEN` (and that it's provided in production .env) will resolve this reliably.

2. Inconsistent Environment Variable Naming (High Severity)

Issue: Environment and secret names vary across the project, causing configuration mix-ups. For example, the OpenWeatherMap key is referred to as `OPENWEATHER_API_KEY` in code but was stored as `OPEN_WEATHER_API_KEY` in GitHub secrets, and the Mapbox token secret was named `MAP_API_TOKEN` while code expected `MAPBOX_TOKEN` ⁴. The front-end expects `VITE_`-prefixed vars (like `VITE_OPENWEATHER_API_KEY` for use in the browser) whereas backend uses non-prefixed names, and in some cases both exist. These inconsistencies led to deployment failures where secrets were present but not picked up by the app ⁴. The project's setup scripts attempt to bridge some differences (e.g., accepting

either `OPENWEATHERMAP_API_KEY` or `OPENWEATHER_API_KEY` for weather) ⁵, but this dual-handling hints at underlying naming confusion.

Affected Components: Build and deployment configurations, environment setup scripts, and any code accessing `process.env`. Notably:

- **GitHub Actions workflows & secrets:** Required remapping secret names to match expected env vars ⁴.
- **Environment setup script** (`scripts/setup-env.cjs`): Falls back between `OPENWEATHERMAP_API_KEY` and `OPENWEATHER_API_KEY` ⁵, showing inconsistent naming. It also maps public-site env like `VITE_MAPBOX_TOKEN` from a base `MAPBOX_TOKEN` if present ⁶, etc.
- **Runtime code:** The shared Mapbox util looks for `MAPBOX_PUBLIC_TOKEN` ² while deployment used `MAPBOX_TOKEN`. Similarly, weather code uses `WEATHER_API_KEY` internally but secrets were under a different name in some contexts.

Impact: Mismatched names cause missing configuration at runtime – e.g., an API key present in the environment under one alias but the code looking for another will behave as if no key is set. This can lead to features silently failing (maps not loading tiles, weather API falling back to fake data, etc.) or deployment processes erroring out. In fact, a major “main issue” in the CI pipeline was that secrets “were there all along, just with slightly different names!” ⁷. This delayed deployment until corrected.

Recommendation: Standardize environment variable names across all components and documentation. Choose a single naming convention (e.g., all keys uppercase with underscores, no ambiguous abbreviations) and apply it uniformly. Update code to use these canonical names, and adjust build/CI scripts accordingly. For frontend, ensure all needed runtime vars are `VITE_`-prefixed and documented. Clean up any redundant or legacy variable usage to avoid confusion (for instance, use **only** `OPENWEATHER_API_KEY` everywhere, and deprecate `OPENWEATHERMAP_API_KEY`). The provided **Secret Name Mapping** document ⁴ is a good reference – these mappings should be codified in the project’s `.env.example` files and setup scripts so that both developers and CI use the same terms. Adding runtime checks or warnings for missing critical env vars (already partially done in `setup-env.cjs` ⁸ ⁹) will also help catch misnaming early.

Evidence: The secret mapping documentation explicitly lists the needed changes to align names ⁴. For example, the code expected `MAPBOX_TOKEN` but the secret was named `MAP_API_TOKEN` ¹⁰. Similarly, the weather API key had an underscore mismatch ¹⁰. The environment setup script shows multiple fallbacks for similarly purposed keys ⁵ ¹¹, confirming that inconsistent naming had to be patched in code. By unifying these, such patches can be removed and the configuration will be less error-prone.

3. Hardcoded API Keys and Secrets in Repository (Critical Severity)

Issue: Multiple API keys, tokens, and credentials are directly present in the source code or committed config files, posing serious security risks. Examples include: a Mapbox public token in React code ¹, an OpenWeatherMap API key and Tessie API token in the Edge Worker’s dev vars ¹², and even Tesla API credentials, ABRP keys, and a Mapbox token in the MCP server’s `.env` ¹³. These values appear to be real or at least formatted like real secrets (e.g., actual Mapbox `pk...` tokens, long random strings for tokens). If this repository is public or ever became public, those secrets would be compromised. Even in a private repo, committing secrets is dangerous due to potential leaks or unauthorized access.

Affected Components:

- **Repository config files:** e.g. `edge-worker/.dev.vars` contains plaintext keys for Mapbox, OpenWeather, Tessie, etc. ¹². The MCP server's `.env` file holds Tesla OAuth client info, refresh tokens, etc. ¹³. These files are under version control.
- **Source code constants:** e.g. `public-site/src/components/Map.jsx` embedding the Mapbox token ¹, iOS `RealAPIEndpoints.swift` embedding placeholder API keys like `"tsk_live_48continental_key"` for Tessie and others ¹⁴ ¹⁵. Even if some are dummy placeholders, they suggest the real values might be similarly handled or easily derivable.
- **Test or example code:** Some docs or examples (like `shared/examples/tesla-api-example.ts`) might contain sample tokens, though the bigger concern is the actual dev/prod config.

Impact: Security is critically impacted. Mapbox tokens (if public scope) could be abused for mapping requests at your expense, OpenWeather and Tessie keys could be used by others (possibly incurring cost or hitting rate limits). The Tesla refresh token and client secret in the MCP `.env` ¹⁶ are particularly sensitive – they could allow an attacker full API access to the Tesla account (including vehicle controls). Committing these secrets violates best practices and likely terms of service of those APIs. Additionally, having secrets scattered in code complicates rotation and environment-specific changes.

Recommendation: Remove all secrets from the code repository immediately. Migrate to a secure secret management strategy: use environment variables (not committed) and/or secret stores (GitHub Secrets for CI, Cloudflare Workers KV/Secrets for edge, etc.). The project already has guidance for GitHub Actions secrets ¹⁷ – ensure those are the only source of truth and that local dev uses a non-committed `.env.local`. The `.dev.vars` and similar files should be excluded from version control (or replaced with sanitized example files). Any secret that has been exposed in git history (e.g., the keys shown in this audit) should be assumed compromised – **revoke and rotate them** (generate new tokens/keys via the respective services). This includes the Tessie API token, Tesla refresh token, Mapbox token, OpenWeather key, etc.

For future development, enforce strict separation: code reads config from env vars, and real values are injected via CI or local `.env` files listed in `.gitignore`. This will prevent accidental commits of secrets. Tools like **BlackBox** (noting there's a `.blackboxrules` file in the repo) might be used to encrypt secrets in the repo if they must be there, but plain text is unacceptable ¹² ¹³.

Evidence: The presence of actual API keys is visible in config files – for instance, the Edge Worker dev vars file includes what appear to be real tokens for Mapbox and Tessie ¹². The MCP `.env` contains a Tesla OAuth refresh token JWT and client secret ¹⁶. These examples underscore the severity: such secrets should never be in source control.

4. OpenWeatherMap Integration – Data Consistency & Unit Handling (Medium Severity)

Issue: The project's weather integration is split and possibly inconsistent across components, leading to incorrect data presentation. The Edge Worker fetches current weather from OpenWeatherMap (OWM) in metric units ¹⁸ ¹⁹, but the frontend expects or displays data as if it were imperial. Specifically, the Edge's `fetchWeatherData` returns temperature in Celsius (`units=metric`) and wind speed in m/s (converted to km/h) ¹⁹. However, the React component that displays weather (`WeatherDisplay.jsx`) treats the data like a direct OWM response and appends "°F" to the temperature and "mph" to wind speed without

conversion ²⁰ ²¹. This mismatch means a temperature of, say, 20°C from the API would be shown as “20°F” in the UI, confusing or misleading users.

Additionally, different parts of the system use different weather sources: the mobile/iOS side uses NOAA's weather.gov (NWS) API (as seen in iOS code) while the web uses OWM. That could result in different readings or conditions being shown on different platforms (e.g., one showing “Clear 68°F” and another “Sunny 20°C” for the same location if not unified). The project doesn't clearly normalize this. The OWM integration itself is only fetching current weather (`.../weather?lat=...` endpoint) and not forecasts, whereas NWS integration might involve forecasts – potential feature divergence.

Affected Components:

- **Edge Worker:** `utils/weather-api.ts` fetches and parses OWM current weather ¹⁸ ¹⁹. It uses metric units and converts wind to km/h. The `WEATHER_API_KEY` env is used here ²².
- **Frontend Web:** `public-site/src/dataSources/weatherApi.js` calls the Edge endpoint but (due to a bug, see Issue 6) often falls back to a sample. When it does get real data, the `WeatherDisplay` component expects OWM's JSON structure (with `main.temp`, `weather[0].description`, etc.) ²³. It displays those values with imperial unit labels ²⁰ ²¹ but does **no unit conversion**.
- **Mobile/iOS:** `RealAPIEndpoints.swift` suggests using Weather.gov for forecasts (NWS endpoints) ²⁴ ²⁵ – which returns Fahrenheit by default. There's a disparity in data source and possibly units between mobile and web.

Impact: Users may see incorrect weather information. For example, the web app might show a temperature inaccurately labeled as °F when it's actually a Celsius value (e.g., “20°F” instead of 68°F). Wind speeds could be off by a factor (km/h shown as mph numbers). Such errors reduce trust in the app's data, especially for an EV road trip where weather is important. The inconsistency between platforms can also be jarring (different apps showing different weather for the same trip).

Recommendation: Unify the weather data handling:

- Decide on a single measurement system for display (likely Fahrenheit and mph for a U.S. audience) and convert all data to that before presentation, or allow the user to select units. If sticking with imperial, either query OWM with `units=imperial` or convert the metric results. **Ensure the labels match the data.** The `WeatherDisplay.jsx` should display °F only if the number is indeed Fahrenheit. Since edge currently gets Celsius, either change the API call to `units=imperial` or convert `data.main.temp` from C to F (and km/h to mph) in the edge response or front-end.
- Standardize weather sources if possible. Using OWM for current conditions and NOAA for forecasts can be done, but ensure they are consistent (e.g., both in the same units and similar formats). It might be simpler to use one service for both current and forecast to avoid discrepancies.
- Test the entire flow: feed known values through edge to UI to confirm, for instance, 0°C shows as 32°F, 10 m/s wind (~22 mph) is labeled ~22 mph, etc.
- On the coding side, adjust `WeatherDisplay`: instead of directly taking `weatherData.main.temp`, use a normalized `weatherData.temperature` in a consistent unit. In fact, the `WeatherDisplay.propTypes` suggests a model where `weatherData.temperature` could be a unified field ²⁶, but currently it's reading raw OWM fields ²³. Refactoring to use a cleaned structure (maybe produced by the edge or by the hook) would help.

Evidence: The edge weather fetch clearly uses metric units and populates a `Weather` object with Celsius and km/h ¹⁹. In contrast, the React display prints an “°F” suffix on the temperature and “mph” on wind with

no conversion ²⁰ ²¹. This indicates an oversight in integration. Aligning these will fix the user-visible errors.

5. Tessie API Real-Time Data Flow Issues (High Severity)

Issue: The integration with Tessie (the Tesla API proxy service) for live vehicle telemetry may not be functioning as intended due to endpoint mismatches and token handling. The app is supposed to stream live vehicle data (state-of-charge, speed, location, etc.) via WebSockets from the Edge Worker, but there are discrepancies in how the frontend and backend establish this connection:

- **WebSocket Endpoint Mismatch:** The frontend uses an environment variable `VITE_WEBSOCKET_ENDPOINT` which is set to a URL ending in `/ws` ²⁷. It then opens `wsEndpoint/<vehicleId>` (appending the VIN or id) as the socket URL ²⁸ ²⁹. However, on the Edge Worker side, the code listens for WebSocket upgrade requests at a different path (`/tesla/vehicle/stream`) ³⁰, not `/ws`. Unless there's an unseen route alias or the `wsEndpoint` environment was specifically crafted to point to the correct path, this suggests a client/server misalignment. For example, if `VITE_WEBSOCKET_ENDPOINT = wss://thewanderingwhittle-edge.workers.dev/ws`, the client will connect to `.../ws/<vin>` - but the Cloudflare worker expects a request to `/tesla/vehicle/stream` to initiate the socket (and likely doesn't handle an arbitrary `/ws` path by default). This could result in the WebSocket connection failing (404 or just not upgrading), meaning no live data comes through.
- **Authentication & Token Use:** Tessie requires an API token (provided via env `TESSIE_API_TOKEN`) and VIN to fetch data. The Edge Worker's *HTTP* endpoint for vehicle data (`GET /api/vehicle`) uses these to retrieve a snapshot, caching if needed ³¹ ³². For real-time streaming, the Edge likely either proxies Tessie's streaming API or uses Tessie's telemetry WebSocket. There is code in `shared/tessie/tessieClient.ts` and `edge-worker/src/tessie-telemetry-stream.ts` for connecting to Tessie's streaming endpoint ³³ ³⁴. But it's unclear if this is fully integrated (the Edge might not currently expose a `/ws` that directly pipes Tessie data). The frontend's `createVehicleDataStream` does attempt to parse incoming messages as Tessie data and transform them ³⁵ ³⁶, so it assumes the server will forward JSON telemetry. If the WebSocket never connects, the frontend will silently keep trying to reconnect without ever receiving data.

Affected Components:

- **Frontend:** `vehicleApi.js` - specifically `createVehicleDataStream()` which builds the WebSocket URL and manages reconnects ²⁸ ²⁹. It relies on `VITE_WEBSOCKET_ENDPOINT`.
- **Edge Worker:** Possibly the Durable Object or Worker script handling `/tesla/vehicle/stream`. In `index.ts`, `handleVehicleStream` is imported and invoked for that path ³⁰. That likely ties into `vehicle-stream.ts` or `tessie-telemetry-stream.ts`. If `handleVehicleStream` ultimately uses `TessieTelemetryStream` from `tessie-telemetry-stream.ts`, it would connect to Tessie's `wss://telemetry.tessie.com/stream/<VIN>` and pipe data. But there's a complexity: Tessie's streaming might require a separate fleet subscription (the code implies use of Tessie's fleet WebSocket with an auth message ³³ ³⁴). Ensuring the Edge Worker actually initiates this and forwards messages to the correct client socket is vital.
- **Environment/Config:** `VITE_WEBSOCKET_ENDPOINT` in `.env.production` is set (to a Cloudflare Workers dev URL) ²⁷, but the Edge Worker must accept connections at that path. If the Worker script was

deployed with a route pattern where `/ws` is mapped to `/tesla/vehicle/stream`, it's not visible in code and would need confirmation.

Impact: If the WebSocket stream is broken, the application will not show live updates of the vehicle (charging status, SOC%, location moves) in real time – undermining one of the interactive aspects of the project (the “live journey” feel). The frontend will likely just show static or periodically polled data. The user might notice that the vehicle position or battery doesn't update unless they refresh or the app falls back to polling via the `/api/vehicle` endpoint (if at all). In effect, a key integration (real-time Tesla data via Tessie) could be nonfunctional, a **high-priority integration bug** for a “road trip tracker” app.

Recommendation: Align the WebSocket communication protocol between front and back:

- **Fix endpoint routing:** Ensure that the path the frontend connects to is handled by the worker. E.g., simplest fix on client side: change `VITE_WEBSOCKET_ENDPOINT` to include the needed route (perhaps `wss://.../tesla/vehicle/stream`). Or modify the Cloudflare worker to accept `/ws` as an alias (since environment suggests `/ws` usage, maybe add a condition: if pathname starts with `/ws/`, extract the vehicleId and invoke the stream handler).
- **Verify end-to-end:** After adjusting, test that a WebSocket connection can be established (no 404 or upgrade error) and that messages flow. Given Tessie's design, the worker might need to send an authorization message on behalf of the client or include the Tessie token; in `TessieTelemetryStream` code, it sends an `{type: 'auth', token: <TESSIE_TOKEN>}` upon connection ³⁴. Confirm the worker does this and then pushes incoming telemetry to the client socket. The client's code already handles JSON messages and transforms them ³⁵ ³⁶.
- **Error handling:** Add logging or UI indication if the WebSocket fails to connect (the client code calls `onError` and eventually marks status as “failed” ³⁷ ³⁸, but the user might not see that). Perhaps have the UI fall back to a polling mode (calling `/api/vehicle` periodically) if streaming is unavailable.
- **Security:** Ensure the WebSocket endpoint is protected if needed. Currently, it seems open (no auth on the client side beyond possibly the HMAC signature used for other endpoints). If that's a concern, consider requiring the same HMAC or a simple auth for the socket connection.

Evidence: The client WebSocket setup clearly expects to use the `/ws` endpoint from config ²⁸, and appends the vehicleId to it ²⁹. Meanwhile, the worker's code listens on a different route ³⁰, indicating a likely discrepancy. Without correction, this is a broken linkage. After remediation, you should observe the console log “Vehicle data stream connected” on the client ³⁹ and see incoming telemetry being logged/applied.

6. API Route Mismatches Between Frontend and Backend (Medium Severity)

Issue: There are cases where the frontend calls an endpoint that doesn't exactly match what the backend expects, causing fallback behavior or failures. A concrete example is the weather API: the React app fetches weather data from `<EdgeWorkerDomain>/weather?lat=...&lon=...` ⁴⁰, but the Cloudflare worker is set up to handle this request at the path `/api/weather` (as indicated by `if (url.pathname === "/api/weather") ... handleWeatherAPI` ⁴¹). Since the client omits `/api`, it will hit a route that likely isn't handled – the worker's default behavior for unknown paths is to try serving static content or return 404. Indeed, `handleStaticFile` would likely return 404 for `/weather` if not found in KV ⁴². The client code does log a warning and falls back to sample data when the fetch is not OK ⁴³. This means in practice

the web app probably never gets real weather from the worker, always using the dummy sample (unless a redirect or alias was set up separately).

Other potential mismatches might exist: e.g., if the frontend expects an endpoint for charging station data or itinerary. We see the worker defines `/api/stations` and `/api/trip` with mock data ⁴⁴ ⁴⁵, and the front-end likely calls similar paths (the code references `EDGE_WORKER_DOMAIN` and uses generic patterns). It's crucial these align. The weather one is confirmed misaligned.

Affected Components:

- **Frontend:** `public-site/src/dataSources/weatherApi.js` uses `EDGE_WORKER_DOMAIN + "/weather"` ⁴⁰. Similarly, if any other `dataSources` or hooks call endpoints, we should verify they include the `/api/` prefix. The weather hook (`useWeatherData`) ultimately invokes `fetchWeatherData` from that module ⁴⁶.
- **Edge Worker:** Route handling in `index.ts` expects certain prefixes (all API routes are under `/api/...` per design) ⁴⁴. There is no handler for bare `/weather`. Unless Cloudflare Workers has a routing rule mapping `/weather` to the script differently (not evident here), this is a miss.

Impact: The immediate impact is that the web app never shows actual live weather from OpenWeatherMap. It will log a warning "using sample data" ⁴³ and display static dummy weather (which might not match reality at all). This undermines the reliability of the weather info for the trip visualization. More broadly, such mismatches can cause features to silently degrade – developers might think data is being pulled from backend when it isn't. Inconsistent use of `/api/` could also lead to CORS issues or broken links if not standardized.

Recommendation: Audit all frontend API calls to ensure they match the backend routes. For weather specifically, update the URL in `weatherApi.js` to `.../api/weather` (the `EDGE_WORKER_DOMAIN` could include the path, or simply append `"/api/weather"` in the fetch call). Confirm that other calls, like for vehicle data (`fetchVehicleData` uses `API_BASE_URL` which defaults to `"/api"` so that one is correct ⁴⁷ ⁴⁸) and charging stations or trip data, are correctly prefixed. If there's a pattern, consider centralizing a config like `API_BASE_URL` for all services to prepend `/api` as needed – in fact, the production `.env` defines `VITE_API_BASE_URL=https://...workers.dev` ⁴⁹ which presumably includes no trailing slash; if the code consistently uses e.g. `fetch(API_BASE_URL + '/weather')`, then including `/api` in that base might be a solution. Just be cautious not to double-up `"/api/api"`.

After fixing the endpoints, test each data fetch. The weather fetch should return real data (and then also adjust units as per Issue 4). Monitor the console or network to ensure 200 OK from the worker instead of fallback.

Additionally, maintain consistency: all backend HTTP endpoints are under `/api/*` for clarity (except maybe websockets). Document this for future contributors. If needed, implement a redirect or alias in the Worker for legacy paths (e.g., treat `/weather` as alias to `/api/weather` to handle older frontends), but since you control both sides, aligning the client is simpler.

Evidence: The code clearly shows the discrepancy – front-end calling `.../weather` ⁴⁰ vs worker listening on `/api/weather` ⁴¹. This is a straightforward fix once identified.

7. Configuration Drift Between Components (Medium Severity)

Issue: Different parts of the project maintain their own configuration constants (API endpoints, tokens, etc.), which have become inconsistent, especially after renaming or environment changes. For instance, the iOS client still references the Cloudflare worker by an old domain (`route-tracker.48continental.workers.dev`)⁵⁰, whereas the web and edge use the new domain (`thewanderingwhittle-edge.workers.dev`). If the old domain is no longer updated, the iOS app would be pointing to a stale or nonexistent backend. Similarly, environment values are duplicated in multiple places (.env files for each sub-project), raising the risk that one gets updated (or renamed) and others don't.

The **MapBox token** is another example: the front-end had an older token hardcoded¹, while `.env.production` contains a new token value⁵¹ presumably for the new account (matching the new MapBox username "thewanderingwhittle"). If the code doesn't consistently use the new token, parts of the system might still be using the deprecated one.

There's also evidence of **development vs production config divergence**: e.g., `.env.local` vs `.env.production` have different endpoint URLs and tokens. The risk is if the app isn't correctly loading the production one in a deployed environment (or if a developer runs the wrong config, hitting production services with dev credentials).

Affected Components:

- **iOS app (Swift client):** `RealAPIEndpoints.swift` has hardcoded base URLs and API keys that may not be current⁵²¹⁵. No environment-based override is evident, so any change requires a code update. For example, `cloudflareBaseURL` still points to the old subdomain⁵³. If the Cloudflare Worker was redeployed to a new domain, the iOS app must update or it will fail to communicate.
- **React Native (if exists) and Web:** Likely rely on the `.env.production` values⁵¹ for endpoints and tokens. If not properly loaded, they might default to local or outdated endpoints.
- **Edge Worker & MCP:** Their env files have some overlapping config (e.g., `MAPBOX_TOKEN` in multiple places). The mapping doc shows how GitHub Actions needed to adapt to new names⁴, indicating the config drift was being corrected, but vigilance is needed to keep them in sync.

Impact: If the iOS app is not updated, features like trip data, map, or weather might not load at all for the end-user (pointing to wrong URLs). This is an integration-breaking issue for that client. Having different API keys in different places can also cause subtle bugs – e.g., using a test Mapbox style in one client and a production style in another could render maps differently. For environment-specific behavior: if production values aren't properly injected (say the Pages site not using the `.env.production` correctly), the live site might inadvertently use placeholder data or dev endpoints. That could mean, for instance, the live site still hitting `localhost` or a wrong domain for the Edge API, failing to load dynamic data.

Recommendation: Centralize and externalize configuration wherever possible. Some steps:

- Use environment files or build configs that are environment-specific, but ensure each platform (web, iOS, RN) is reading from a source that can be updated without code changes. For iOS, consider moving to a plist or config file that can be replaced per build configuration, or fetch remote config on launch. At minimum, document clearly that `RealAPIEndpoints.swift` needs updating whenever backend endpoints change (and prefer not to embed secrets there – see Issue 3).

- Align all components to the new project naming. Search the codebase for “48continental” – update those to “thewanderingwhittle” or new identifiers as appropriate (domains, keys, etc.). The iOS references to 48continental should be replaced with the current ones (if the Cloudflare worker now uses “thewanderingwhittle-edge”, update the URL and any associated tokens).
- Eliminate duplicate definitions of the same config. For example, instead of having `MAPBOX_TOKEN` in both Edge and MCP .env, consider if one is needed (the Edge might only need it if it calls Mapbox APIs itself; otherwise the frontends should use their own tokens). Similarly for weather API keys – ideally one service (Edge) calls OWM and frontends get weather via Edge, so only Edge needs the key. If multiple components truly need the same secret, ensure they all reference a single configured value (for instance, through CI injecting the same secret to each sub-component’s env). The **SECRET_NAME_MAPPING.md** suggests using a single GitHub secret for a key and mapping it to multiple env vars if needed ¹⁷ – implement those mappings in CI and verify each environment picks up the value.
- Regularly test each client (web, iOS, etc.) against the staging/prod environment to catch config drift. A deployment checklist should include verifying that environment variables are loaded as expected and endpoints are correct.

Evidence: The iOS config still points to the old worker domain ⁵³ and uses placeholder API keys for services that likely have new values ¹⁴. Meanwhile, the web’s production env has new tokens/domains ⁵¹. This mismatch confirms that without reconciliation, the iOS app would not function fully with the current backend.

8. Overly Complex Agent System Architecture (Low Severity)

Issue: The project outlines an ambitious “enterprise-grade” hierarchical agent architecture (MCP – Model Context Protocol – with CEO, SLTs, Dept Heads, Managers, and dozens of specialized MCP agents) ⁵⁴ ⁵⁵. In practice, much of this appears conceptual or only partially implemented. The Cloudflare Worker’s MCP integration currently only includes a few handlers (Sequential Thinking, Filesystem, Browser Tools) ⁵⁶, and the MCP server (and related Orchestrator) is relatively simple, mostly handling WebSocket broadcasts for agent communication ⁵⁷ ⁵⁸. This discrepancy between the grand design and the actual code can introduce unnecessary complexity and potential inefficiencies. The layering of agents (with message passing between them) might be overkill at this stage, leading to confusion and more points of failure or maintenance burden. In short, the architecture may be **over-engineered** relative to the current needs, which can impede scalability and clarity instead of aiding it.

Affected Components:

- **Documentation vs. Code:** The `docs/agent-architecture.md` paints a very complex hierarchy ⁵⁴ ⁵⁹. If developers try to adhere to this, they may spend time writing layers of agents that don’t yield proportional benefit.
- **MCP Server (Node) and Cloudflare MCP integration:** There is an `AgentOrchestrator` WebSocket server in Node that simply broadcasts messages to all clients after a basic auth check ⁵⁷ ⁵⁸. Similarly, the Cloudflare Durable Object `SyncServiceDurableObject` does broadcast to all connected WebSockets ⁶⁰. This duplicative functionality (one in Node, one in Edge) suggests the architecture hasn’t decided on a single approach for agent comms and might be unnecessarily splitting focus. If both are used, keeping them in sync or combined is non-trivial.
- **Agents’ responsibilities:** Many agent roles (e.g., Weather MCP vs MapBox MCP vs Tesla MCP) likely overlap with straightforward function calls that could be done without an agent framework. There’s a risk of

spending resources on inter-agent messaging when a direct service call would suffice, thereby affecting performance as more agents spin up or communicate.

Impact: While not directly user-facing, this complexity can impact development velocity and system performance at scale. The more layers and processes, the more potential for race conditions or message loss if not carefully handled (imagine dozens of MCP agents sending messages through a central orchestrator – without proper filtering or routing, you get a broadcast storm). It can also consume memory/CPU: e.g., each agent might maintain state that could have been centralized. In terms of scalability, a single WebSocket hub broadcasting everything to everyone (as the orchestrators do now ⁵⁸ ⁶⁰) will not scale well if, say, 20+ agents are actively chatting – it's essentially an $O(n^2)$ message pattern. Moreover, new contributors might be intimidated or misled by the architecture docs, thinking they need to implement a full corporate-like hierarchy, which is unusual for an application of this scope.

Recommendation: Simplify and rationalize the agent system. If the multi-agent concept is crucial (perhaps for AI behaviors), consider flattening it to the minimal necessary interactions. For instance, if you have an agent to fetch weather and an agent to plan charging stops, they might communicate, but this could be done with a direct function call or a focused message, rather than going up and down a hierarchy. Keep the “CEO/manager” metaphor as internal documentation if desired, but ensure the code implementation is lean: possibly a single orchestrator that coordinates a few functional modules (without dozens of trivial intermediary agents).

For the communications, avoid redundant channels. Decide if Cloudflare Durable Objects will handle agent sync (preferred if agents are distributed between edge requests) or if the Node MCP server will (if running persistent processes). They both doing similar broadcasts is unnecessary – perhaps retire the Node WebSocket orchestrator if everything can run on the Edge (simplifying deployment), or vice versa if you need long-lived processes on a server. The key is to prevent double-handling and reduce overhead.

Also, enforce scope on agent messaging: implement filtering or addressing (not every message should go to every agent/client as it does now ⁶⁰). This will improve performance and clarity – e.g., use message types or topics so that, say, a “Weather agent” only processes weather-related messages rather than sifting through everything.

In summary, treat the current agent architecture as a prototype; profile it under load (multiple agents sending frequent messages) to identify bottlenecks, and refactor accordingly. It may be that a simpler pub-sub or even direct method calls are sufficient until the system truly needs the complexity. This will enhance scalability by not overburdening it with an unneeded complex message bus.

Evidence: The documentation graph ⁵⁴ ⁵⁵ shows an extremely granular hierarchy, but the actual code implementing MCP is minimal. For instance, only three server handlers are registered in the Cloudflare MCP index ⁵⁶, and the orchestrators simply broadcast all messages to all participants ⁵⁸. This gap suggests over-engineering on paper that isn't realized in code – an opportunity to simplify in practice.

9. Code Redundancy and Logic Maintainability (Low Severity)

Issue: There are instances of duplicated logic and placeholder code that could make maintenance harder. While not causing immediate bugs, these patterns reduce clarity: for example, the vehicle data

transformation is implemented in both the Edge Tessie client and again in the frontend (with slightly different conditions) ⁶¹ ⁶² ⁶³ ⁶⁴ . Maintaining two versions of “transform Tesla API data to our format” means updating both if Tesla’s data structure changes or if you add a new field. Similarly, weather data simulation is done in both Edge (`generateFallbackWeatherData` ⁶⁵ ⁶⁶) and frontend (`generateWeatherData` in `weatherApi.js` ⁶⁷ ⁶⁸), using different algorithms. Test and debug code (like `MapDebugPanel`, `MapDebug.jsx`) and dev-only features remain in the repository, which is fine, but some are in active code paths (e.g., `mapUtils` vs custom validation duplication, etc.).

Additionally, some **stub implementations** are still present even in presumably production code paths: the Edge’s `/api/trip` and `/api/stations` return hardcoded mock data ⁶⁹ ⁷⁰ ⁷¹ . If the frontend relies on these, users are seeing dummy trip progress or station info. While understandable during development, these need replacement or gating for a production deployment.

Affected Components:

- **Data transformation functions:** `edge-worker/src/tessie-client.ts` vs `public-site/src/api/vehicleApi.js` have overlapping “transform to standard format” logic ⁶³ ⁶⁴ and ⁷² ⁶² .
- **Weather fallback:** Edge’s `generateFallbackWeatherData()` ⁶⁵ ⁶⁶ vs frontend’s `generateWeatherData()` ⁷³ ⁶⁸ produce weather samples differently (one based on trig deterministic by coord/time, one random seasonal). This inconsistency might cause confusion if one side expects the other’s pattern.
- **Map coordinate handling:** There was an introduction of a centralized `ensureMapboxFormat` utility, but initially custom code existed; ensuring only one method is used is important (looks like they did integrate the utility in `Map.jsx` ⁷⁴ ⁷⁵ , so that’s good).
- **Stubbed endpoints:** `handleTripAPI` and `handleStationsAPI` in the Edge are clearly placeholders with static data ⁶⁹ ⁷⁰ ⁷¹ , marked by comments. If the project is moving to production/MVP, these should either be implemented or at least documented to the user (e.g., “demo data”). Leaving them as-is could mislead users or testers about the state of actual functionality (e.g., trip progress might always show the same route or states).

Impact: Redundant code can lead to bugs when one side changes and the other doesn’t. For instance, if we add a new field (say, tire pressure monitoring) to the vehicle data, and update the Edge transform but forget the frontend transform, the WebSocket path might send a field the frontend ignores or misplaces. It’s extra work to keep in sync. The stubbed data, if not replaced, provides incorrect info (the trip progress might not actually reflect the real journey state or might never mark completion, etc.). While this is a lower severity compared to outright failures, it affects the quality and accuracy of the application and increases technical debt.

Recommendation: Refactor to DRY (Don’t Repeat Yourself) where feasible:

- For data transforms (Tesla/Tessie data): consider moving the transformation entirely to the backend edge worker, so that the frontend only ever deals with already-normalized data. In fact, `handleTessieVehicle` already does transform and returns `transformedData` ⁷⁶ , and the WebSocket streaming code on the frontend calls `transformTessieData()` on incoming messages ³⁵ ³⁶ . It might be better to push that transformation to the server side as well (i.e., have the worker send already-transformed telemetry JSON). That way, the shape of vehicle data is defined in one place. If frontend still needs to handle raw data (for direct Tessie API calls or something), you could share a utility via the `shared/` folder or an NPM package so that both use the exact same logic.
- Remove or clearly toggle dummy data: For `/api/trip` and `/api/stations` , if there isn’t time to

implement fully, guard them with a check like “if in development mode, return mock, otherwise indicate not available” or document that it’s static. Ideally, implement these to return real data (e.g., trip progress could be derived from stored itinerary data and current state, and stations could come from a charging station API). At minimum, note in the UI if data is simulated. The same goes for weather: if fallback is being used, maybe label it “(estimated)” in the UI to differentiate from real data.

- Clean up development artifacts as you stabilize: The presence of components like `MapDebugPanel`, extra console logs (`console.log('Map component using centralized utility...')` ⁷⁴) is useful during development, but trimming these out or wrapping them so they don’t appear in production builds will make the app feel more polished and possibly improve performance slightly (less console chatter). Keep debug capabilities in a way that can be toggled (e.g., via a `debug=true` URL param, which is partially implemented ⁷⁷).

By reducing redundancy and cleaning stubs, you make the codebase leaner and easier to update. This also ties into agent complexity (Issue 8) – simpler code with single-responsibility components will be easier to maintain in the long run.

Evidence: The duplication of transformation logic is visible in code – compare the structure of `transformToStandardFormat` in the Edge Tessie client ⁶³ ⁶⁴ with `transformTessieData` in the frontend ⁷² ⁶² – both map Tesla/Tessie fields to a local schema, albeit with minor differences in property names (e.g., `estimatedRange` vs not included on frontend). This duplication is a maintainability smell. Likewise, the presence of obvious placeholder data in API responses ⁷⁰ ⁷¹ highlights where future work or cleanup is needed.

10. Error Handling Gaps and Resilience (Low Severity)

Issue: While the code handles many error cases, there are scenarios where the error handling could be improved to ensure robustness. For example, in `handleTessieVehicle`, if Tessie API is not configured (missing token or VIN), it currently throws an error that gets caught and returns a 503 with “Failed to fetch vehicle data” ⁷⁸ ⁷⁹. There’s no fallback in this case (since it’s first-time and no cache), so the client simply wouldn’t get vehicle info. Similarly, if the Tessie streaming WS fails to connect (token invalid or vehicle offline), the client code will retry a few times then give up ⁸⁰ ³⁸ – but the user isn’t explicitly notified that live data is unavailable. For weather, if the API key is missing, the worker uses a deterministic fake data which is a clever fallback ⁸¹ ¹⁹, but the user might not know it’s not real. Also, some error logging is done (e.g., printing to console) which in a serverless environment is good, but aggregating or monitoring those would be necessary to catch issues (especially if something like Tessie token expiry happens unexpectedly, etc.).

Affected Components:

- **Edge Worker** `handleTessieVehicle`: Returns cached data on Tessie failure if available, otherwise a 503 error JSON ⁸² ⁷⁹. No attempt is made to use the Tesla API as a fallback if Tessie fails (the code has a separate `handleTeslaVehicle` for direct Tesla API, but it’s not invoked here). If Tessie credentials are wrong or its service is down, the user might be left with nothing unless a manual fallback strategy is initiated.

- **WebSocket streaming (frontend)**: After max reconnection attempts, it logs an error and marks status as “failed” ⁸³, but unless the UI is observing that `onStatusChange` callback to alert the user, it might just silently stop trying.

- **General UI/UX for errors:** Many errors are only in console (`console.error(...)`). For a public-facing app, it might be better to surface some of these to users in a friendly way (e.g., “Live data temporarily unavailable” or “Failed to load weather, using last known data”). Currently, placeholders like Weather and Trip data will just show static values without context.

Impact: The system might not gracefully handle certain failures or might leave the user unaware that data is stale or incomplete. While not catastrophic, this can lead to confusion. For instance, if Tessie token was accidentally not set in production env, the map would never show the car – but the user would only see an initial battery icon maybe and then nothing updating, with no explanation. Clear messaging would improve user experience during outages or config issues. Additionally, from a development perspective, certain errors might be missed if only logged to console (which, on Cloudflare, means Workers KV logs or similar – one needs to actively check those). If not monitored, a failing integration could go unnoticed in prod for a while.

Recommendation: Strengthen error handling and feedback loops:

- Implement user-facing notifications for critical data fetch failures. For example, if the vehicle data endpoint returns an error or empty, have the frontend detect that and display something like “Unable to load vehicle data at this time.” The Weather hook could set an `weatherError` state (it does, but perhaps display it in the UI) ⁸⁴ . The Map component already sets `mapError` on token issues and shows a console error ⁸⁵ ⁸⁶ – maybe extend that to render a visible message.

- Provide fallback strategies where possible. If Tessie fails, one idea is to try the Tesla API (since the code has Tesla OAuth ability via `handleTeslaVehicle`). Perhaps not automatically (since waking a Tesla via API repeatedly isn’t ideal), but as a manual admin option it could be useful. At minimum, cache should perhaps persist longer – currently `lastKnownVehicleData` is in-memory for the Cloudflare Worker instance ⁸⁷ . If the worker restarts (which can happen), that cache is gone. Storing last known data in KV or Durable Object could allow returning slightly older data instead of none. This might be overkill for MVP, but worth noting.

- Monitor and log: Ensure Cloudflare Worker errors (like any `console.error`) are being captured by some logging/alerting (maybe via Workers Analytics Engine or sending to an external logging service). This helps detect when integrations fail (e.g., if OpenWeatherMap starts 403ing due to an expired key, you’d see repeated warnings “returned status X, using fallback” ⁸⁸). With monitoring, you can react before users report issues.

- Test error scenarios: intentionally run the system with a bad Mapbox token, or simulate Tessie being unreachable, to see how the app behaves, and improve the handling accordingly. The goal is that the app should degrade gracefully: use cached or simulated data with clear labeling, and recover automatically when possible.

Evidence: The current approach shows both good and improvable patterns. The Tessie vehicle handler does a smart thing by using cached data if Tessie fails ⁸⁹ ⁹⁰ , but ultimately can still end up returning an error if nothing cached ⁷⁹ . The weather fetch falls back to generated data on any failure ¹⁸ ⁹¹ , which is great – yet the UI doesn’t distinguish it as fake. By implementing the above recommendations, the system’s robustness and transparency will increase, rounding out the quality of the user experience.

Patterns and Systemic Issues: Across these findings, a common theme is **consistency** – in naming, data formats, and config – which is essential for a multi-component project. Many issues arose from slight inconsistencies (env var names, unit systems, endpoint paths). Another theme is **over-extension** – having

more complex structures (agent hierarchy, duplicated logic) than currently needed. Addressing these by simplifying and unifying will both improve current stability and make future development easier. Security hygiene (removing secrets, handling tokens properly) is another systemic concern that needs continual attention, as it underpins user trust and project integrity. By tackling the critical items first (secret management, integration alignment) and then iterating on quality improvements (refactoring redundancy, enhancing error handling), the project will be well-positioned to deliver a reliable and secure experience for users tracking the Wandering Whittle's journey.

1 74 75 77 85 86 **Map.jsx**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/src/components/Map.jsx

2 **mapboxConfig.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/shared/mapbox/mapboxConfig.ts>

3 **map-fix-task-prompt.md**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/map-fix-task-prompt.md

4 7 10 17 **SECRET_NAME_MAPPING.md**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/docs/SECRET_NAME_MAPPING.md

5 6 8 9 11 **setup-env.cjs**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/scripts/setup-env.cjs>

12 **.dev.vars**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/.dev.vars>

13 16 **.env**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/mcp-server/.env>

14 15 24 25 50 52 53 **RealAPIEndpoints.swift**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/ios-client/Sources/Configuration/RealAPIEndpoints.swift>

18 19 65 66 81 88 91 **weather-api.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/src/utis/weather-api.ts>

20 21 23 26 **WeatherDisplay.jsx**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/src/components/WeatherDisplay.jsx

22 30 31 32 41 42 44 45 69 70 71 76 78 79 82 87 89 90 **index.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/src/index.ts>

27 49 51 **.env.production**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/.env.production

28 29 35 36 37 38 39 47 48 61 62 72 80 83 **vehicleApi.js**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/src/api/vehicleApi.js

33 34 **tessie-telemetry-stream.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/src/tessie-telemetry-stream.ts>

40 43 67 68 73 **weatherApi.js**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/src/dataSources/weatherApi.js

46 84 **useWeatherData.js**

https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/48Continental_Starter/public-site/src/hooks/useWeatherData.js

54 55 59 **agent-architecture.md**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/docs/agent-architecture.md>

56 **index.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/src/mcp/index.ts>

57 58 **agentOrchestrator.js**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/mcp-server/src/orchestrator/agentOrchestrator.js>

60 **syncService.js**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/shared/agent-coordination/syncService.js>

63 64 **tessie-client.ts**

<https://github.com/JW-Flo/AWhittleWandering/blob/770d868251e9122fc42ea43db8934dad0cb8ba81/edge-worker/src/tessie-client.ts>